

TP : travailler en équipe avec les pull requests

Module : Versioning et gestion de versions décentralisé (RNCP 39608, bloc BC03)

Format : matinée en distanciel, de 9 h 00 à 12 h 00, en équipe de cinq.

Objectifs du TP

À la fin de la matinée, on sait :

1. contribuer à un dépôt partagé sans jamais pousser directement sur `main` ;
2. nommer une branche et rédiger des messages de commit selon une convention d'équipe ;
3. ouvrir une pull request, la faire relire, la corriger et la fusionner ;
4. relire le code d'un camarade et rédiger une review utile ;
5. exploiter le rapport de SonarQube Cloud pendant une review ;
6. mettre sa branche à jour avec `main` et résoudre un conflit.

Le projet

On travaille sur **Mémo Git**, un petit site statique qui affiche des pages rédigées en Markdown. La page d'accueil liste les pages, identifiées par leur titre de niveau 1. Un clic sur un titre affiche la page, et un lien permet de revenir à l'accueil. Il n'y a pas de back-end : tout le rendu est fait dans le navigateur.

Le site est fonctionnel. Pendant le TP, on le fait évoluer en développant des features choisies dans un catalogue (annexe B). Chaque feature correspond à une issue GitHub numérotée de #1 à #12.

Déroulé (indicatif) de la matinée

Horaire	Séquence	Contenu
9 h 00 à 9 h 20	Lancement	Appel, consignes, attribution des numéros E1 à E5
9 h 20 à 10 h 30	Partie 1	Préparer son poste et réaliser sa première pull request
10 h 30 à 10 h 45	Point d'étape 1	Bilan de la partie 1, choix des premières features
10 h 45 à 11 h 15	Partie 2	Développer une feature, la faire relire avec SonarQube Cloud
11 h 15 à 11 h 25	Point d'étape 2	Bilan de la partie 2, choix des secondes features
11 h 25 à 11 h 45	Partie 3	Développer une seconde feature, synchroniser sa branche, résoudre les conflits
11 h 45	Conclusions	Bilan collectif, remise de la fiche de TD

Celui qui termine une partie en avance passe à la suivante sans attendre le point d'étape.

La fiche de TD

Chaque élève remplit sa propre fiche de TD, fournie à part. Les questions sont signalées dans ce document par la mention **Fiche de TD**, suivie du numéro de la question. On y répond au moment où elle apparaît : la plupart des réponses demandent de copier un résultat ou d'observer quelque chose à cet instant précis.

La fiche n'est pas versionnée dans le dépôt du projet. On la remet à l'enseignant à 12 h 00, selon les modalités qu'il indique au lancement.

Les rôles et la rotation des reviewers

Au lancement, l'enseignant attribue à chacun un numéro, de E1 à E5. Ce numéro sert à deux choses : la ligne à compléter dans `CONTRIBUTEURS.md` en partie 1, et la désignation des reviewers.

Dans une pull request, on joue l'un de ces deux rôles :

1. **l'auteur** écrit le code, ouvre la pull request, répond aux remarques et fusionne quand toutes les conditions sont réunies ;
2. **le reviewer** relit le code, le teste, commente et décide d'approuver ou de demander des modifications.

Les reviewers changent à chaque partie. Le tableau indique, pour chaque auteur, qui relit ses pull requests :

Auteur	Partie 1	Partie 2	Partie 3
E1	E2	E3 et E4	E5
E2	E3	E4 et E5	E1
E3	E4	E5 et E1	E2
E4	E5	E1 et E2	E3
E5	E1	E2 et E3	E4

Sur la matinée, chacun relit chacun des quatre autres. En partie 2, les deux reviewers doivent approuver avant la fusion. Une pull request ouverte en partie 2 et pas encore fusionnée à 11 h 15 garde ses reviewers de la partie 2.

Les gestes GitHub

Toutes les manipulations de l'interface de GitHub sont décrites une seule fois, dans l'annexe A, sous la forme de gestes numérotés de G1 à G9. Dans la suite du document, on renvoie simplement au geste, par exemple : « on ouvre la pull request [G3] ».

Quand une pull request est-elle prête à fusionner ?

Une pull request ne se fusionne que si les quatre conditions suivantes sont réunies :

1. chaque reviewer désigné a approuvé la pull request ;
2. toutes les conversations ouvertes pendant la review sont résolues ;
3. la quality gate de SonarQube Cloud est validée (**Passed**) ;
4. GitHub ne signale aucun conflit avec `main`.

Les conditions 1 (au moins une approbation) et 2 sont imposées par une règle du dépôt : GitHub bloque la fusion tant qu'elles ne sont pas remplies. Les autres reposent sur la discipline de l'équipe, et le reviewer les vérifie.

C'est l'auteur qui fusionne sa pull request [G9].

Lancement

1. Adresse du dépôt (<https://github.com/trentindev/memo-git>) et celle du projet SonarQube Cloud ;
2. vérifie que chacun a reçu l'invitation à collaborer sur le dépôt.

On inscrit son numéro et ces deux adresses en tête de sa fiche de TD.

Partie 1 : préparer son poste et réaliser sa première pull request (9 h 20 à 10 h 30)

Objectif : chacun ajoute son nom dans le fichier `CONTRIBUTEURS.md` en passant par une pull request. Le contenu de la modification est volontairement simple, pour se concentrer sur le circuit complet : branche, commit, push, pull request, review, fusion.

1.1 Vérifier ses outils

Sous Windows, toutes les commandes de ce document se tapent dans **Git Bash**. Sous Linux et macOS, on utilise le terminal habituel.

```
git --version
node --version
```

Le projet demande Node.js 22.13 ou supérieur. Si la version affichée est inférieure, on installe la version LTS depuis <https://nodejs.org> avant d'aller plus loin.

On vérifie son identité Git :

```
git config --global user.name
git config --global user.email
```

Si une des deux commandes n'affiche rien, on la configure :

```
git config --global user.name "Prénom Nom"
git config --global user.email "adresse@exemple.fr"
```

L'adresse e-mail doit être une de celles déclarées sur son compte GitHub : c'est elle qui permet à GitHub de rattacher les commits à son profil.

On indique à Git qu'un `git pull` doit faire une fusion (merge) quand la branche locale et la branche distante ont divergé. Sans ce réglage, Git s'arrête dans ce cas et demande de choisir une stratégie :

```
git config --global pull.rebase false
```

1.2 Accéder au dépôt

On accepte l'invitation au dépôt [G1]. Sans cette étape, tous les pushes sont refusés.

Le mot de passe du compte GitHub n'est pas accepté pour les opérations Git. On utilise une de ces deux méthodes :

1. sous Windows, Git for Windows inclut Git Credential Manager : au premier push, une fenêtre de connexion à GitHub s'ouvre, on choisit la connexion par le navigateur et on valide ;
2. sur tous les systèmes, si GitHub CLI est installé, on lance `gh auth login`, on choisit **GitHub.com**, puis **HTTPS**, on répond **Yes** à la question qui propose d'authentifier Git avec ses identifiants GitHub, puis on choisit **Login with a web browser**.

Si on utilise déjà une clé SSH ou un jeton d'accès personnel qui fonctionne, on le garde.

1.3 Cloner le projet et le lancer

On se place dans le dossier où on range ses projets :

```
git clone https://github.com/<proprietaire>/memo-git.git
cd memo-git
npm ci
npm start
```

On ouvre <http://localhost:8080> dans le navigateur, on parcourt les quatre pages, puis on arrête le serveur avec `Ctrl+C` dans le terminal.

On lance ensuite la vérification complète du projet :

```
npm run check
```

Cette commande enchaîne trois contrôles : ESLint (règles de qualité du JavaScript), Prettier (mise en forme) et les tests unitaires. Les trois doivent réussir avant chaque push, pendant toute la matinée.

On regarde comment le dépôt local connaît le dépôt distant :

```
git remote -v
```

Fiche de TD : Q1.

1.4 Lire les règles du projet

On lit `README.md` et `CONTRIBUTING.md` en entier. Ils décrivent la structure du code, le format des noms de branche et la convention des messages de commit. L'annexe C détaille cette convention.

Fiche de TD : Q2.

1.5 Créer sa branche et modifier le fichier

L'issue #13 « Compléter la liste des contributeurs » est commune à toute l'équipe. On crée sa branche à partir d'un `main` à jour. Dans les commandes, on remplace `prenom` par son prénom en minuscules, sans accent :

```
git switch main
git pull
git switch -c docs/13-contributeur-prenom
```

On ouvre `CONTRIBUTEURS.md` et on remplace **uniquement** la ligne « À compléter » située sous son numéro par : `Prénom Nom, @compte-github`. On ne touche à aucune autre ligne.

On vérifie ce qui a changé :

```
git status
git diff
```

`git diff` ne doit montrer qu'une ligne supprimée (préfixe `-`) et une ligne ajoutée (préfixe `+`).

1.6 Commiter et pousser

```
git add CONTRIBUTEURS.md
git commit -m "docs: ajouter Prénom à la liste des contributeurs" -m "Refs #13"
git push -u origin docs/13-contributeur-prenom
```

Chaque option `-m` ajoute un paragraphe au message : le premier `-m` donne la ligne de titre, le second ajoute une ligne de pied de message qui référence l'issue.

Fiche de TD : Q3.

1.7 Ouvrir la pull request et la faire relire

On ouvre la pull request [G3] avec ces éléments :

1. titre : `docs: ajouter Prénom à la liste des contributeurs` ;
2. description : on complète le modèle, en remplaçant la ligne `Closes #` par `Refs #13` ;
3. reviewer : celui de la colonne « Partie 1 » du tableau de rotation [G4].

Fiche de TD : Q4.

Quand on est désigné comme reviewer, on relit la pull request concernée :

1. on lit le rapport de SonarQube Cloud [G8] : la pull request ne modifie qu'un fichier Markdown, que SonarQube n'analyse pas, la quality gate doit donc être validée ;
2. on vérifie dans l'onglet **Files changed** qu'une seule ligne a changé, celle du bon numéro ;
3. on vérifie le titre de la pull request et le message du commit (onglet **Commits**) ;
4. on commente au moins une ligne [G5], même pour confirmer que tout est correct ;
5. on soumet sa review [G6] : **Approve** si tout est conforme, **Request changes** sinon.

L'auteur traite les éventuelles remarques [G7], puis fusionne sa pull request [G9].

Fiche de TD : Q5.

1.8 Mettre à jour son dépôt local

Une fois sa pull request fusionnée, on revient sur `main` et on récupère les fusions de toute l'équipe :

```
git switch main
git pull
git log --oneline -8
```

Fiche de TD : Q6.

GitHub a supprimé la branche distante après la fusion, mais le dépôt local garde une référence `origin/docs/13-contributeur-prenom` vers cette branche. On retire les références aux branches distantes qui n'existent plus :

```
git fetch --prune
```

On supprime ensuite sa branche locale, devenue inutile :

```
git branch -d docs/13-contributeur-prenom
```

Git refuse avec le message `error: the branch 'docs/13-contributeur-prenom' is not fully merged.`

Fiche de TD : Q7.

On force la suppression :

```
git branch -D docs/13-contributeur-prenom
```

Si on a terminé avant 10 h 30, on lit le catalogue des features (annexe B) et on passe directement à la partie 2.

Point d'étape 1 (10 h 30 à 10 h 45)

Chacun indique où il en est. Les pull requests de la partie 1 non fusionnées sont terminées après le point d'étape, avant de commencer la partie 2.

On choisit sa première feature, dans l'ordre suivant : E2, E4, E3, E5, E1. Celui qui choisit en premier dispose du catalogue complet.

Partie 2 : développer une feature et la faire relire (10 h 45 à 11 h 15)

Objectif : livrer une première feature du catalogue en suivant le circuit complet, avec deux reviewers et le rapport de SonarQube Cloud.

2.1 Choisir et prendre en charge sa feature

Chaque feature du catalogue indique un niveau et une zone. On choisit en tenant compte de deux règles :

1. on choisit une feature de son niveau, ou du niveau au-dessus ;
2. en partie 2, deux élèves ne travaillent pas en même temps sur deux features de la même zone.

La zone indique les fichiers que la feature modifie. La seconde règle garantit qu'aucune pull request de la partie 2 n'entre en conflit avec une autre. On traitera les conflits en partie 3.

On prend en charge l'issue correspondante [G2].

Fiche de TD : Q8.

2.2 Créer sa branche

On part toujours d'un `main` à jour. Exemple pour la feature F5 (issue #5) :

```
git switch main
git pull
git switch -c feat/5-compteur-pages
```

Le nom de branche suit le format `type/numero-description` du fichier `CONTRIBUTING.md`. Le catalogue propose un nom pour chaque feature.

Fiche de TD : Q9.

2.3 Développer en commits courts

On découpe son travail en plusieurs commits. Chaque commit contient une seule modification logique et laisse le projet dans un état qui fonctionne. Pour une feature de code, on commite en général séparément le code et les tests.

Pendant le développement, on vérifie régulièrement le résultat dans le navigateur avec `npm start`, et on lance `npm run check` avant chaque push.

Exemple pour la feature F5 :

```
git add src/js/views.js
git commit -m "feat(accueil): afficher le nombre de pages disponibles"
git add tests/views.test.js
git commit -m "test(accueil): tester le libellé du nombre de pages"
npm run check
git push -u origin feat/5-compteur-pages
```

Si Prettier signale des fichiers mal mis en forme, `npm run format` les corrige. On commite cette correction avant de pousser.

On affiche les commits de sa branche qui ne sont pas encore dans `main` :

```
git log --oneline main..HEAD
```

Fiche de TD : Q10.

2.4 Ouvrir la pull request

On ouvre la pull request [G3] :

1. titre : au format des messages de commit, par exemple `feat(accueil): afficher le nombre de pages disponibles` ;

2. description : on complète le modèle et on indique `Closes #5` ;

3. reviewers : les deux élèves de la colonne « Partie 2 » [G4].

Le dépôt fusionne les pull requests en **squash and merge** : tous les commits de la branche sont regroupés en un seul commit sur `main`, et ce commit reprend le titre de la pull request. C'est pour cela que le titre doit respecter la convention.

2.5 Relire une pull request

Quand on est désigné comme reviewer, on suit cet ordre.

1. **Le rapport de SonarQube Cloud** [G8]. On note le statut de la quality gate et chaque problème signalé. L'annexe D explique comment lire ce rapport.
2. **Le respect de l'issue**. On relit les critères d'acceptation de la feature dans le catalogue, et on vérifie que la pull request les remplit tous, sans rien ajouter d'autre.
3. **Le test en local**. On récupère la branche de l'auteur et on vérifie le résultat :

```
git fetch origin
git switch feat/5-compteur-pages
npm run check
npm start
```

`git switch` crée automatiquement une branche locale qui suit la branche distante du même nom. Après le test, on revient sur `main` avec `git switch main`.

4. **La lecture du code**, dans l'onglet **Files changed**. Lisibilité, nommage, cohérence avec le reste du projet, présence de tests pour les features de code.
5. **L'historique**, dans l'onglet **Commits** : nom de la branche, messages de commit conformes à l'annexe C.

On rédige ses remarques ligne par ligne [G5]. Une remarque utile dit ce qui pose problème, pourquoi, et propose une solution. Quand la correction tient en quelques lignes, on la propose sous forme de suggestion. Une review porte sur le code, jamais sur la personne.

On soumet sa review [G6]. Si on demande des modifications, on le dit clairement avec **Request changes**.

Fiche de TD : Q11 et Q12.

2.6 Répondre à la review

Quand on est l'auteur et qu'on reçoit une review :

1. on répond à chaque remarque, soit en la corrigeant, soit en expliquant pourquoi on ne la retient pas ;
2. on applique les suggestions directement dans GitHub [G7], ou on corrige en local puis on pousse un nouveau commit sur la même branche ;
3. on relance la review des reviewers concernés [G4].

Si on a appliqué une suggestion dans GitHub, la branche distante contient un commit que la branche locale n'a pas. On le récupère avant de continuer à travailler en local :

```
git pull
```

Fiche de TD : Q13.

Le reviewer vérifie les corrections, résout les conversations qu'il a ouvertes [G7] et approuve. Quand les quatre conditions sont réunies, l'auteur fusionne [G9].

2.7 Mettre à jour son dépôt local

```
git switch main
git pull
git fetch --prune
git branch -D feat/5-compteur-pages
```

Point d'étape 2 (11 h 15 à 11 h 25)

Chacun présente sa feature en une minute. On note les problèmes signalés par SonarQube Cloud dans les pull requests de l'équipe.

On choisit sa seconde feature, dans l'ordre inverse du point d'étape 1 : E1, E5, E3, E4, E2.

Partie 3 : seconde feature et résolution de conflits (11 h 25 à 12 h 00)

Objectif : livrer une seconde feature alors que `main` a évolué depuis la création de sa branche, et intégrer ces évolutions en résolvant les conflits.

3.1 Choisir sa seconde feature

La règle des zones de la partie 2 ne s'applique plus. On choisit de préférence une feature dont la zone a déjà été modifiée par une pull request fusionnée en partie 2, ou une feature de la même zone qu'un camarade : c'est ce qui produit des conflits, et c'est le sujet de cette partie.

On prend en charge l'issue [G2], on crée sa branche à partir d'un `main` à jour, on développe et on commite comme en partie 2.

3.2 Synchroniser sa branche avec `main`

Pendant qu'on développe, des camarades fusionnent leurs pull requests dans `main`. Avant d'ouvrir sa pull request, on intègre ces évolutions dans sa branche :

```
git fetch origin
git merge origin/main
```

`git fetch origin` télécharge les nouveaux commits du dépôt distant sans modifier ses fichiers. `git merge origin/main` fusionne ensuite la version distante de `main` dans la branche courante.

Deux cas sont possibles.

Git fusionne sans conflit. Il crée un commit de fusion. On relance `npm run check`, puis on pousse.

Git signale un conflit, par exemple :

```
Auto-merging src/content/pages.json
CONFLICT (content): Merge conflict in src/content/pages.json
Automatic merge failed; fix conflicts and then commit the result.
```

Fiche de TD : Q14.

3.3 Résoudre le conflit

On liste les fichiers en conflit :

```
git status
```

Ils apparaissent dans la section `Unmerged paths`, avec la mention `both modified`, ou `both added` quand les deux branches ont créé le même fichier.

On ouvre chaque fichier en conflit. Git y a écrit les deux versions, encadrées par des marqueurs :

```
[
  "introduction.md",
  "depot-local.md",
<<<<<<< HEAD
  "annuler.md",
=====
  "depot-distant.md",
>>>>>>> origin/main
  "branches.md",
  "pull-requests.md"
]
```

La partie entre `<<<<<<< HEAD` et `=====` est la version de sa branche. La partie entre `=====` et `>>>>>>>` `origin/main` est la version de `main`. Les lignes situées hors des marqueurs ne sont pas en conflit : Git les a déjà fusionnées.

Résoudre un conflit ne consiste pas à choisir une des deux versions au hasard : on écrit le contenu final qui conserve le travail des deux côtés. Dans l'exemple, les deux pages doivent rester publiées, et le fichier doit rester un JSON valide, avec une virgule après chaque entrée sauf la dernière :

```
[
  "introduction.md",
  "depot-local.md",
  "depot-distant.md",
  "annuler.md",
  "branches.md",
  "pull-requests.md"
]
```

Le fichier ne doit plus contenir aucun marqueur. Quand le conflit porte sur du code, on vérifie que le résultat combine bien les deux comportements : chaque fonction ajoutée de part et d'autre doit être présente une seule fois, et chaque import nécessaire doit être présent.

On vérifie, puis on termine la fusion :

```
npm run check
git add src/content/pages.json
git commit --no-edit
git push
```

`--no-edit` valide directement le message de fusion proposé par Git, sans ouvrir d'éditeur. Si on se retrouve malgré tout dans l'éditeur Vim, on tape `Échap`, puis `:wq`, puis `Entrée`.

Pour abandonner une fusion en conflit et revenir à l'état d'avant :

```
git merge --abort
```

Fiche de TD : Q15.

3.4 Ouvrir la pull request et la faire relire

On ouvre la pull request [G3] avec le reviewer de la colonne « Partie 3 » [G4], et on suit le même circuit qu'en partie 2.

Si un camarade fusionne une pull request pendant la review et que GitHub affiche **This branch has conflicts that must be resolved**, on refait la synchronisation de la section 3.2, puis on pousse. La pull request se met à jour toute seule.

Le reviewer vérifie avec une attention particulière les commits de fusion : une résolution de conflit est du code comme un autre, et c'est l'endroit où l'on perd le plus facilement le travail d'un camarade.

Fiche de TD : Q16.

3.5 Pour aller plus loin (facultatif)

Cette section s'adresse à ceux qui ont livré leurs deux features.

Au lieu de fusionner `main` dans sa branche, on peut rejouer ses commits par-dessus la dernière version de `main` :

```
git fetch origin
git rebase origin/main
```

En cas de conflit, on le résout dans le fichier, puis `git add <fichier>` et `git rebase --continue`, autant de fois que nécessaire. Le rebase réécrit l'historique de la branche : ses commits reçoivent de nouvelles empreintes, et le push simple est refusé. On pousse alors avec :

```
git push --force-with-lease
```

Contrairement à `--force`, `--force-with-lease` refuse d'écraser la branche distante si quelqu'un d'autre y a poussé depuis le dernier `fetch`. Ce push forcé est possible sur une branche de feature, mais la règle du dépôt l'interdit sur `main`.

Fiche de TD : Q17.

Conclusions (12 h 00)

On affiche ensemble l'historique de `main` :

```
git switch main
git pull
git log --oneline --graph -20
```

Chacun répond à la dernière question de la fiche, puis la remet à l'enseignant.

Fiche de TD : Q18.

Annexe A : les gestes GitHub

Chaque geste est décrit une seule fois. L'interface de GitHub est en anglais : les libellés sont donnés tels qu'ils apparaissent à l'écran.

G1 : accepter l'invitation au dépôt

L'invitation arrive par e-mail. On peut aussi l'ouvrir directement à l'adresse

`https://github.com/<proprietaire>/memo-git/invitations`. On clique sur **Accept invitation**. On arrive alors sur la page du dépôt.

G2 : prendre en charge une issue

Dans l'onglet **Issues** du dépôt, on ouvre l'issue de la feature choisie. Dans la colonne de droite, sous **Assignees**, on clique sur **assign yourself**. On ajoute un commentaire en bas de l'issue, par exemple « Je prends cette feature, branche feat/5-compteur-pages », puis on clique sur **Comment**.

Une issue qui a déjà une personne assignée est prise : on en choisit une autre.

G3 : ouvrir une pull request

Après le premier push d'une branche, deux chemins mènent au formulaire de création :

1. le terminal affiche un lien `https://github.com/<proprietaire>/memo-git/pull/new/<branche>` après la ligne `Create a pull request for '<branche>' on GitHub by visiting: ;`
2. sur la page du dépôt, un bandeau jaune propose le bouton **Compare & pull request**.

En haut du formulaire, on vérifie les deux branches : **base** doit être `main`, **compare** doit être sa branche.

On saisit le titre. La description est préremplie avec le modèle du projet

(`.github/pull_request_template.md`) : on complète chaque rubrique et on coche les cases avec `[x]`.

Dans la colonne de droite, on désigne les reviewers [G4] et on s'assigne soi-même sous **Assignees**.

On clique sur **Create pull request**. La flèche à droite de ce bouton propose aussi **Create draft pull request** : un brouillon est visible par l'équipe mais ne peut pas être fusionné. On le passe en relecture plus tard avec le bouton **Ready for review**.

G4 : désigner ou relancer les reviewers

Dans la colonne de droite de la pull request, on clique sur **Reviewers** (ou sur l'icône d'engrenage à côté), on tape le nom du compte du camarade, on le sélectionne, puis on clique en dehors de la liste pour enregistrer.

Après avoir corrigé suite à une review, on relance le reviewer : à côté de son nom, dans la même rubrique, on clique sur l'icône **Re-request review** (deux flèches en cercle). Le reviewer reçoit une notification.

G5 : commenter une ligne et proposer une suggestion

Dans l'onglet **Files changed**, on survole la ligne à commenter et on clique sur l'icône + bleue qui apparaît à gauche. Pour commenter plusieurs lignes, on clique sur le numéro de la première ligne puis, en maintenant **Maj**, sur celui de la dernière.

On rédige le commentaire. Pour proposer une correction, on clique sur l'icône **Add a suggestion** (fichier avec un plus et un moins) : un bloc **suggestion** contenant les lignes sélectionnées apparaît, et on le modifie pour écrire la version proposée.

Pour le premier commentaire, on clique sur **Start a review**. Pour les suivants, on clique sur **Add review comment**. Les commentaires restent en attente, visibles de soi seul, jusqu'à la soumission de la review [G6]. Le bouton **Add single comment** publie au contraire le commentaire immédiatement : on l'évite pendant une review, pour envoyer toutes ses remarques en une seule notification.

G6 : soumettre sa review

Dans l'onglet **Files changed**, on clique sur le bouton **Review changes**, en haut à droite (selon la version de l'interface, il peut s'intituler **Submit review**). On rédige un commentaire de synthèse, puis on choisit le type de review :

1. **Comment** : des remarques, sans se prononcer sur la fusion ;
2. **Approve** : le reviewer approuve la fusion ;
3. **Request changes** : le reviewer demande des modifications avant la fusion.

On valide avec **Submit review**. L'auteur d'une pull request ne peut pas approuver la sienne : pour lui, seul **Comment** est disponible.

G7 : répondre à une review

Dans l'onglet **Conversation**, chaque remarque du reviewer apparaît sous forme de conversation. On y répond avec le champ **Reply**.

Une suggestion s'applique avec le bouton **Commit suggestion**. Pour appliquer plusieurs suggestions en un seul commit, on clique sur **Add suggestion to batch** pour chacune, puis sur **Commit suggestions**. GitHub demande un message de commit : on respecte la convention de l'annexe C.

Quand la remarque est traitée, le reviewer qui l'a ouverte clique sur **Resolve conversation**. Dans ce projet, la fusion est bloquée tant qu'une conversation reste ouverte.

G8 : lire le rapport de SonarQube Cloud dans une pull request

Quelques minutes après chaque push sur la branche d'une pull request, SonarQube Cloud publie son résultat à trois endroits :

1. dans l'onglet **Conversation**, un commentaire de SonarQube Cloud résume l'analyse : statut de la quality gate (**Quality Gate passed** ou **Quality Gate failed**), nombre de nouveaux problèmes, security hotspots, duplication ; le lien **See analysis details on SonarQube Cloud** ouvre le détail ;
2. en bas de l'onglet **Conversation**, dans la liste des vérifications, la ligne **SonarCloud Code Analysis** indique le statut ; le lien **Details** ouvre le rapport dans l'onglet **Checks** ;
3. dans l'onglet **Files changed**, les problèmes apparaissent sous forme d'annotations sur les lignes concernées.

Le commentaire est mis à jour à chaque nouvelle analyse : on vérifie toujours qu'il correspond au dernier push.

G9 : fusionner une pull request

En bas de l'onglet **Conversation**, le cadre de fusion récapitule l'état des reviews, des conversations, des vérifications et des conflits. Si une condition imposée par la règle du dépôt n'est pas remplie, GitHub indique laquelle et désactive la fusion.

Quand les quatre conditions du document sont réunies, on clique sur **Squash and merge**. GitHub propose un titre de commit composé du titre de la pull request suivi de son numéro, par exemple `feat(accueil): afficher le nombre de pages disponibles (#19)`. On vérifie ce titre, puis on clique sur **Confirm squash and merge**.

La pull request passe au statut **Merged**. Le dépôt est configuré pour supprimer automatiquement la branche distante après la fusion. Si la description contient `closes #numero`, l'issue est fermée automatiquement.

Annexe B : catalogue des features

Douze features sont proposées, une par issue GitHub (#1 à #12). Chaque élève en livre au moins deux sur la matinée.

Le niveau indique la difficulté :

1. **niveau 1** : contenu Markdown ou CSS, aucune programmation nécessaire ;
2. **niveau 2** : modification ciblée du JavaScript, avec un test ;
3. **niveau 3** : évolution d'un module JavaScript, avec plusieurs tests et une attention à la complexité du code.

Feature	Intitulé	Niveau	Zone	Branche proposée
F1	Page « Annuler une modification »	1	Contenu	<code>feat/1-page-annuler</code>
F2	Page « Glossaire »	1	Contenu	<code>feat/2-page-glossaire</code>
F3	Page « Travailler avec un dépôt distant »	1	Contenu	<code>feat/3-page-depot-distant</code>
F4	Documentation de l'architecture	2	Documentation	<code>docs/4-architecture</code>

Feature	Intitulé	Niveau	Zone	Branche proposée
F5	Nombre de pages sur l'accueil	2	Accueil	feat/5-compteur-pages
F6	Tri alphabétique des pages	2	Chargement	feat/6-tri-alphabetique
F7	Titres sans syntaxe Markdown sur l'accueil	2	Moteur Markdown	fix/7-titres-sans-markdown
F8	Mise en forme du code	1	Styles	feat/8-style-code
F9	Mode sombre automatique	2	Styles	feat/9-mode-sombre
F10	Listes numérotées	3	Moteur Markdown	feat/10-listes-numerotees
F11	Navigation page précédente et page suivante	3	Navigation	feat/11-navigation-pages
F12	Filtre de recherche sur l'accueil	3	Accueil	feat/12-recherche-accueil

Les fichiers de chaque zone :

Zone	Fichiers modifiés
Contenu	src/content/pages.json et un nouveau fichier dans src/content/
Documentation	docs/architecture.md (nouveau) et README.md
Accueil	src/js/views.js (fonction renderHome) et tests/views.test.js (nouveau)
Chargement	src/js/pages.js, tests/pages.test.js et README.md
Moteur Markdown	src/js/markdown.js et tests/markdown.test.js
Styles	src/styles/main.css
Navigation	src/js/views.js (fonction renderPage), src/js/app.js et src/styles/main.css

Pour toutes les pages de contenu, on n'utilise que la syntaxe Markdown prise en charge par le projet, décrite dans README.md. Les commandes Git citées doivent avoir été testées dans un terminal.

F1 : page « Annuler une modification »

On crée src/content/annuler.md et on l'ajoute dans pages.json, juste après depot-local.md.

Critères d'acceptation :

1. le titre de niveau 1 est « Annuler une modification » et il apparaît sur l'accueil ;
2. la page présente quatre situations, chacune dans une section de niveau 2 avec un bloc de code : abandonner les modifications d'un fichier (`git restore <fichier>`), retirer un fichier de la zone de préparation (`git restore --staged <fichier>`), corriger le dernier commit tant qu'il n'est pas poussé (`git commit --amend`), annuler un commit déjà partagé en créant un commit inverse (`git revert <commit>`) ;
3. la page précise, pour `git restore <fichier>`, que les modifications abandonnées ne sont pas récupérables.

F2 : page « Glossaire »

On crée `src/content/glossaire.md` et on l'ajoute à la fin de `pages.json`.

Critères d'acceptation :

1. le titre de niveau 1 est « Glossaire » et il apparaît sur l'accueil ;
2. la page définit au moins huit termes, dont : branche, commit, conflit, dépôt distant, fusion, pull request, review, zone de préparation ;
3. les termes sont présentés en liste à puces, par ordre alphabétique, au format **terme** : définition ;
4. chaque définition tient en une ou deux phrases.

F3 : page « Travailler avec un dépôt distant »

On crée `src/content/depot-distant.md` et on l'ajoute dans `pages.json`, juste après `depot-local.md`.

Critères d'acceptation :

1. le titre de niveau 1 est « Travailler avec un dépôt distant » et il apparaît sur l'accueil ;
2. la page présente `git remote -v`, `git fetch`, `git pull`, `git push` et `git push -u origin <branche>`, chacune avec un bloc de code ;
3. une section explique la différence entre `git fetch` (téléchargement sans modification de la branche courante) et `git pull` (téléchargement puis intégration dans la branche courante) ;
4. la page se termine par un lien vers la page des branches : `[les branches](#/page/branches)`.

F4 : documentation de l'architecture

On crée `docs/architecture.md` et on ajoute un lien vers ce fichier dans la section « Structure du projet » du `README.md`.

Critères d'acceptation :

1. le document décrit le rôle de chacun des cinq modules de `src/js/` en deux à quatre phrases ;
2. il décrit, étape par étape, ce qui se passe entre le clic sur un titre de l'accueil et l'affichage de la page, en citant les fonctions appelées (`parseRoute`, `renderPage`, `renderMarkdown` ...) ;
3. il explique pourquoi le site a besoin du manifeste `pages.json` ;
4. il explique comment le moteur Markdown empêche l'injection de HTML.

F5 : nombre de pages sur l'accueil

Critères d'acceptation :

1. sous le titre « Pages disponibles », un paragraphe indique le nombre de pages : « 4 pages publiées » ;
2. le singulier est respecté : « 1 page publiée » ;
3. quand il n'y a aucune page, le message existant « Aucune page publiée. » reste le seul affiché ;
4. le libellé est produit par une fonction exportée `formatPageCount(count)` dans `views.js`, testée dans un nouveau fichier `tests/views.test.js` (au moins les cas 1 et 4).

F6 : tri alphabétique des pages

Critères d'acceptation :

1. l'accueil affiche les pages dans l'ordre alphabétique de leur titre, quel que soit l'ordre du manifeste ;
2. le tri respecte les règles du français : « Écrire » se place entre « Dépôt » et « Fusion » ; on utilise `Intl.Collator` avec la locale `fr`, ou `localeCompare` avec les mêmes paramètres ;
3. le tri est réalisé par une fonction exportée `sortByTitle(pages)` de `pages.js`, qui renvoie un nouveau tableau sans modifier celui reçu ;
4. le test existant « conserve l'ordre du manifeste » est remplacé par un test du nouvel ordre, et un test vérifie le cas des accents ;
5. la phrase du `README.md` qui parle de « la position souhaitée dans la liste » est mise à jour.

F7 : titres sans syntaxe Markdown sur l'accueil

Constat : une page dont le titre est `# Les commandes **essentielles**` apparaît sur l'accueil avec les astérisques. La fonction `extractTitle` renvoie le texte brut du titre, sans interpréter la syntaxe en ligne. On reproduit le problème avant de le corriger, en créant temporairement une page de test, qu'on ne commite pas.

Critères d'acceptation :

1. `extractTitle` renvoie le texte sans syntaxe : `# Les commandes **essentielles**` donne `Les commandes essentielles`, `# La commande `git switch`` donne `La commande git switch`, `# Lire [Pro Git](https://git-scm.com/book/fr/v2)` donne `Lire Pro Git` ;
2. le traitement est fait par une fonction exportée `stripInline(text)` dans `markdown.js` ;
3. un test couvre chacun des trois exemples ci-dessus.

F8 : mise en forme du code

Critères d'acceptation :

1. le code en ligne a un fond coloré (`var(--color-surface)`), une marge intérieure horizontale et des coins arrondis ;
2. les blocs de code ont une bordure de 1 pixel (`var(--color-border)`) et des coins arrondis ;
3. la taille du texte du code vaut 0,9 fois celle du texte courant (`0.9em`) ;
4. le code situé dans un bloc ne reçoit pas le fond et la marge du code en ligne (sélecteur `.markdown-body`)

`pre_code`);

5. aucune couleur n'est écrite en dur : on n'utilise que les variables de `:root`.

F9 : mode sombre automatique

Critères d'acceptation :

1. quand le système de l'utilisateur est réglé en mode sombre, le site s'affiche en mode sombre ; on utilise la requête média `@media (prefers-color-scheme: dark)` qui redéfinit les variables de `:root` ;
2. on ajoute `color-scheme: light dark;` dans `:root`, pour que le navigateur adapte aussi ses propres éléments (barres de défilement, champs de formulaire) ;
3. le contraste entre le texte et le fond est d'au moins 4,5 pour 1 (niveau AA des WCAG) pour le texte courant, les liens et le texte du pied de page ; on le vérifie avec l'outil de contraste des outils de développement du navigateur ;
4. pour tester sans changer le réglage de son système, dans Chrome : outils de développement (F12), puis menu de commandes (Ctrl+Maj+P), puis commande **Show Rendering**, puis option **Emulate CSS media feature prefers-color-scheme**.

F10 : listes numérotées

Critères d'acceptation :

1. une ligne au format `1. texte` (un à neuf chiffres, un point, au moins une espace) produit un élément `<li>` dans une liste `<ol>` ; les lignes consécutives forment une seule liste ;
2. si le premier numéro n'est pas 1, la liste reçoit l'attribut `start` : une liste qui commence par `3.` produit `<ol start="3">` ;
3. passer d'une ligne `- texte` à une ligne `1. texte` ferme la liste à puces et ouvre une liste numérotée, et inversement ;
4. au moins quatre tests couvrent ces cas, plus le cas d'un paragraphe suivi d'une liste numérotée ;
5. aucune fonction ne dépasse le seuil de complexité cognitive signalé par SonarQube Cloud (règle `javascript:S3776`, seuil de 15) : si `handleTextLine` devient trop complexe, on la découpe.

F11 : navigation page précédente et page suivante

Critères d'acceptation :

1. en bas de chaque page, un bloc de navigation propose un lien vers la page précédente et un lien vers la page suivante, dans l'ordre de l'accueil ; chaque lien affiche le titre de la page visée ;
2. la première page n'a pas de lien « précédente », la dernière n'a pas de lien « suivante » ;
3. le bloc est un élément `<nav>` avec l'attribut `aria-label="Pages précédente et suivante"` ;
4. le calcul des pages voisines est fait par une fonction exportée et testée, qui reçoit la liste des pages et l'identifiant (`slug`) de la page courante.

F12 : filtre de recherche sur l'accueil

Critères d'acceptation :

1. au-dessus de la liste, un champ `<input type="search">` associé à un `<label>` « Filtrer les pages » ;
2. la liste se filtre à chaque frappe (événement `input`) : seules restent les pages dont le titre contient le texte saisi ;
3. la comparaison ignore la casse et les accents : « depot » trouve « Travailler dans un dépôt local » ; on peut utiliser `normalize('NFD')` puis supprimer les caractères de la catégorie Unicode `\p{M}` ;
4. si aucune page ne correspond, un message indique : « Aucune page ne correspond à « texte saisi ». » ;
5. la comparaison est faite par une fonction exportée `matchesquery(title, query)` dans `views.js`, testée dans un nouveau fichier `tests/views.test.js` (casse, accents, texte vide).

Annexe C : guide des messages de commit

Pourquoi une convention

L'historique d'un projet se lit bien plus souvent qu'il ne s'écrit : pendant une review, pour comprendre l'origine d'un bug, pour rédiger les notes de version. Une convention commune rend cet historique lisible par toute l'équipe, et lisible par des outils, qui peuvent par exemple générer automatiquement un journal des modifications ou calculer le prochain numéro de version.

Le projet suit la spécification **Conventional Commits 1.0.0**.

La structure d'un message

`type(portée): description`

Corps facultatif, séparé du titre par une ligne vide.

Pied de message facultatif, par exemple : `Closes #12`

La première ligne est le titre. C'est elle qu'affichent `git log --oneLine` et l'interface de GitHub.

Le type

Type	Usage	Exemple
<code>feat</code>	Nouvelle fonctionnalité ou nouveau contenu visible par l'utilisateur	<code>feat(accueil): ajouter un filtre de recherche</code>
<code>fix</code>	Correction d'un bug	<code>fix(markdown): retirer la syntaxe en ligne des titres</code>
<code>docs</code>	Documentation du projet	<code>docs: décrire l'architecture des modules</code>
<code>style</code>	Mise en forme du code, sans changement de comportement	<code>style: appliquer Prettier aux tests</code>
<code>refactor</code>	Restructuration du code, sans changement de comportement	<code>refactor(markdown): découper handleTextLine</code>

Type	Usage	Exemple
<code>test</code>	Ajout ou correction de tests	<code>test(pages): couvrir le tri des titres accentués</code>
<code>chore</code>	Maintenance : dépendances, configuration	<code>chore: mettre à jour ESLint</code>

Deux pièges fréquents :

1. `style` ne désigne pas la feuille de style : une modification de CSS qui change l'apparence du site est un `feat` (nouvelle apparence) ou un `fix` (correction d'un affichage) ;
2. dans ce projet, une nouvelle page du site est un `feat`, car elle est visible par l'utilisateur ; `docs` est réservé à la documentation destinée aux développeurs (`README.md`, `CONTRIBUTING.md`, `docs/`).

La portée

La portée, facultative, précise la partie du projet concernée, entre parenthèses et sans espace :

`feat(accueil): ...`. Dans ce projet, on utilise les noms suivants : `accueil`, `pages`, `markdown`, `navigation`, `styles`, `contenu`.

La description

1. elle commence par un verbe à l'infinitif : « ajouter », « corriger », « supprimer » ;
2. elle commence par une minuscule et ne se termine pas par un point ;
3. elle décrit ce que fait le commit, pas ce qu'on a fait pendant la journée ;
4. le titre complet tient en 50 caractères si possible, recommandation de la documentation officielle de Git, et ne dépasse jamais 72 caractères, règle retenue dans ce projet.

Le corps

Le corps est facultatif. On l'ajoute quand le titre ne suffit pas à comprendre **pourquoi** la modification est nécessaire : le comment se lit dans le code. On le sépare du titre par une ligne vide et on revient à la ligne avant 72 caractères.

Le pied de message

Le pied de message relie le commit à une issue :

1. `Closes #12` ferme l'issue #12 quand le commit, ou la pull request qui le contient, est fusionné dans la branche par défaut ; GitHub accepte aussi `close`, `closed`, `fix`, `fixes`, `fixed`, `resolve`, `resolves`, `resolved` ;
2. `Refs #13` crée seulement un lien vers l'issue, sans la fermer.

Un changement qui casse la compatibilité se signale par un point d'exclamation après le type (`feat!: ...`) ou par un pied de message commençant par `BREAKING CHANGE:`.

Le contenu d'un commit

Un bon message ne rattrape pas un mauvais découpage. Un commit contient une seule modification logique et laisse le projet dans un état qui fonctionne : on doit pouvoir le décrire en un titre sans utiliser « et ».

Exemples

Message	Problème
<code>modifs</code>	Pas de type, description vide de sens
<code>feat: Ajout de la recherche.</code>	Majuscule, point final, forme nominale au lieu de l'infinitif
<code>fix: corriger le titre et ajouter le mode sombre</code>	Deux modifications logiques dans un commit
<code>style(css): ajouter le mode sombre</code>	<code>style</code> ne désigne pas la CSS
<code>feat(accueil): ajouter un filtre de recherche</code>	Conforme

Rédiger un message en plusieurs lignes

Chaque option `-m` ajoute un paragraphe :

```
git commit -m "fix(markdown): retirer la syntaxe en ligne des titres" -m "Les titres contenant du gras s'affichaient avec les astérisques sur l'accueil." -m "Closes #7"
```

Sans option `-m`, `git commit` ouvre l'éditeur configuré dans Git pour rédiger le message librement.

Le lien avec les pull requests

Le dépôt fusionne en squash and merge : sur `main`, chaque pull request devient un seul commit, dont le message reprend le titre de la pull request. Le titre de la pull request suit donc les mêmes règles qu'un message de commit. Les messages des commits de la branche restent consultables dans l'onglet **Commits** de la pull request.

Annexe D : SonarQube Cloud en bref

Ce que fait l'outil

SonarQube Cloud, édité par SonarSource, réalise une **analyse statique** du code : il examine le code source sans l'exécuter, en le comparant à un ensemble de règles. Il ne remplace pas le reviewer humain. Il le décharge des vérifications mécaniques (variable inutilisée, expression régulière dangereuse, fonction trop complexe) pour qu'il se concentre sur ce qui demande du jugement : le respect de l'issue, la lisibilité, la pertinence de la solution.

L'analyse automatique

Dans ce TP, SonarQube Cloud fonctionne en **analyse automatique** : il lit le code directement dans le dépôt GitHub, sans aucun pipeline CI/CD à écrire. L'analyse se déclenche à chaque push sur `main` et à chaque push sur la branche d'une pull request. Le fichier `.sonarcloud.properties`, à la racine du dépôt, indique où se trouvent le code source (`src`) et les tests (`tests`).

Cette méthode a des limites : elle ne calcule pas la couverture de tests et ne donne pas accès aux journaux d'analyse. Pour aller plus loin, on la remplace par une analyse lancée depuis GitHub Actions, comme dans le TD SonarQube Cloud du module.

Sur l'offre gratuite, l'analyse des pull requests n'est disponible que pour les pull requests qui ciblent la branche principale. C'est le cas de toutes les pull requests de ce TP.

Le vocabulaire

Un **problème** (issue, à ne pas confondre avec une issue GitHub) est un écart par rapport à une règle. Chaque règle a un identifiant, par exemple `javascript:S3776`, qui permet de retrouver son explication. Un problème est rattaché à une ou plusieurs **qualités logicielles** :

Qualité	Question posée
Sécurité (Security)	Le code contient-il une vulnérabilité exploitable ?
Fiabilité (Reliability)	Le code peut-il produire un résultat faux ou planter ?
Maintenabilité (Maintainability)	Le code est-il difficile à lire ou à faire évoluer ?

Pour chaque qualité touchée, la gravité prend une de ces valeurs, de la plus forte à la plus faible : **Blocker**, **High**, **Medium**, **Low**, **Info**.

Un **security hotspot** est un code sensible du point de vue de la sécurité. Ce n'est pas forcément une faille : une personne doit le revoir et décider s'il est sûr ou s'il faut le corriger.

Le **nouveau code** est le code ajouté ou modifié. Dans une pull request, c'est tout ce qui diffère de la branche cible, et seuls les problèmes introduits par la pull request sont signalés.

La quality gate

La quality gate est un ensemble de conditions que le nouveau code doit respecter. Le verdict est binaire : **Passed** ou **Failed**. L'offre gratuite utilise la quality gate « Sonar way », non modifiable :

Condition sur le nouveau code	Seuil
Fiabilité	Note A : aucun nouveau bug
Sécurité	Note A : aucune nouvelle vulnérabilité
Maintenabilité	Note A : dette technique limitée
Security hotspots	Tous revus
Couverture de tests	80 % minimum

Condition sur le nouveau code	Seuil
Lignes dupliquées	3 % maximum

Deux précisions :

1. les conditions sur la couverture et la duplication sont ignorées tant que le nouveau code compte moins de 20 lignes ;
2. l'analyse automatique ne fournit pas de donnée de couverture : sans donnée, la condition de couverture n'est pas évaluée et ne fait pas échouer la quality gate.

Traiter un problème signalé

Décision	Quand l'utiliser
Corriger le code	Le problème est réel : c'est le cas le plus fréquent
Accepter (Accept)	Le problème est réel, mais on décide de ne pas le corriger maintenant, avec une justification écrite
Faux positif (False positive)	La règle se trompe dans ce contexte précis, avec une justification écrite

Pendant ce TP, on corrige le code. Si on pense être face à un faux positif, on en discute avec son reviewer dans la pull request, et on fait valider la décision par l'enseignant.

Où consulter les résultats

Dans chaque pull request [G8], et sur la page du projet SonarQube Cloud dont l'enseignant communique l'adresse au lancement. Le dépôt étant public, cette page se consulte sans compte. Les onglets les plus utiles sont **Summary** (synthèse), **Issues** (liste des problèmes) et **Security Hotspots**.

Annexe E : sources pour aller plus loin

Git :

1. Pro Git, en français, chapitres 3 (les branches) et 5 (Git distribué) : <https://git-scm.com/book/fr/v2>
2. Documentation de `git commit`, section DISCUSSION sur la forme des messages : <https://git-scm.com/docs/git-commit>
3. Conventional Commits 1.0.0, en français : <https://www.conventionalcommits.org/fr/v1.0.0/>

GitHub :

1. À propos des pull requests : <https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests/about-pull-requests>
2. Relire une pull request : <https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/reviewing-changes-in-pull-requests/reviewing-proposed-changes-in-a-pull-request>
3. Lier une pull request à une issue (mots-clés de fermeture) : <https://docs.github.com/en/issues/tracking-your-work-with-issues/using-issues/linking-a-pull-request-to-an-issue>
4. Méthodes de fusion des pull requests : <https://docs.github.com/en/pull-requests/collaborating-with-pull>

[-requests/incorporating-changes-from-a-pull-request/about-pull-request-merges](#)

5. Les rulesets : <https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-rulesets/about-rulesets>

SonarQube Cloud :

1. Analyse automatique : <https://docs.sonarsource.com/sonarqube-cloud/analyzing-source-code/automatic-analysis>
2. Quality gates : <https://docs.sonarsource.com/sonarqube-cloud/standards/managing-quality-gates/introduction-to-quality-gates>
3. Règles, qualités logicielles et gravités : <https://docs.sonarsource.com/sonarqube-cloud/digging-deeper/rules>
4. Analyse avec GitHub Actions (étape suivante, côté CI/CD) : <https://docs.sonarsource.com/sonarqube-cloud/analyzing-source-code/ci-based-analysis/github-actions-for-sonarcloud>